Rapport de Projet – Soutenance 1

« SPECTRALIS »

Participants :
Florent Delbey
Oren Guetta
Tao Brouta-Besset
Julien Dos Santos

EPITA Promo 2029

IMGLOGo.jpeg

Table des matières

1 Rôle et mission de chaque membre **2**
1.1 Florent Delbey (Chef de Groupe & Analyse Spectrale) 2
1.2 Oren Guetta 2
1.3 Tao Brouta-Besset 2
1.4 Julien Dos Santos 3

2 Objectifs, Planification des tâches et Attendus **3**

3 Difficultés & Choix technologiques et Algorithmiques **6**
3.1 Analyse Spectrale - Florent 6
3.2 Synthèse Granulaire - Oren 6
3.3 Entrées/Sorties Audio & Pipeline - Tao 7
3.4 Système & Rendu - Julien 8

4 Bibliographie Détaillée du Projet **11**

1 Rôle et mission de chaque membre

1.1 Florent Delbey (Chef de Groupe & Analyse Spectrale)

En tant que chef de projet, j'assure la coordination de l'équipe, la planification des tâches et l'intégration continue de nos modules. Sur le plan technique, je suis le développeur principal du moteur d'analyse spectrale. Ma mission est de concevoir et d'implémenter l'algorithme de Transformée de Fourier Rapide (FFT) permettant de passer du signal temporel au domaine fréquentiel en temps réel, afin d'alimenter le rendu visuel et de guider les modulations du moteur.

1.2 Oren Guetta

Au sein du projet SPECTRALIS, j'interviens en tant qu'**Expert Traitement de Signal & Synthèse Granulaire**. Ma mission principale est de concevoir le **moteur de synthèse**, qui constitue le cœur créatif du logiciel. Je suis chargé de transformer un flux audio continu en une texture granulaire malléable, en découpant le signal en micro-fragments (*grains*) pour permettre des manipulations temporelles et fréquentielles.

1.3 Tao Brouta-Besset

Responsable de la gestion des entrées/sorties audio, je conçois et implémente le pipeline reliant la carte son aux différents modules de traitement du projet. Mon rôle principal est de mettre en place le callback audio temps réel, c'est à dire la fonction appelée par le système dès que de nouveaux échantillons doivent être fournis au périphérique de sortie. À l'aide de la bibliothèque `cpal`, j'assure la configuration du périphérique audio, la création du flux de sortie et le remplissage continu des buffers audio, tout en respectant les contraintes de performance et de latence du temps réel. Ce travail garantit que les algorithmes de synthèse et d'analyse peuvent être exécutés de manière fluide et stable, en fournissant un lien fiable entre le moteur de traitement interne et le matériel audio.

1.4 Julien Dos Santos

Dans le cadre de ce projet, ma mission principale a été la conception et l'implémentation de l'interface graphique de l'application ainsi que du site web de présentation du projet. J'ai travaillé en collaboration avec les membres en charge de l'analyse FFT, de la synthèse granulaire et du pipeline audio pour définir des interfaces claires entre les différents modules. L'objectif était de proposer une visualisation lisible du spectre en temps réel, un panneau de contrôle intuitif pour les paramètres de synthèse, et une expérience utilisateur globale cohérente.

En parallèle de la GUI applicative, j'ai pris en charge la réalisation du site web du projet. Ce site a pour but de présenter le contexte, les objectifs, l'équipe et les livrables, tout en offrant une vitrine soignée pour la soutenance. J'ai ainsi défini la structure multi-page (accueil, projet, équipe, ressources), choisi une charte graphique sombre avec des accents néon en lien avec l'esthétique de l'interface audio, et intégré les différents contenus fournis par le groupe. Mon rôle s'est donc situé à l'interface entre les aspects techniques et la mise en valeur du travail de l'équipe.

2 Objectifs, Planification des tâches et Attendus

Module / Responsable	Tâche (Code Rust)	Soutenance 1 (23-27 mars)
Analyse Spectrale	Algorithme FFT	70%
Florent	Gestion fenêtres et magnitudes	40%
Synthèse Granulaire	Archi du Buffer Circulaire	50%
Oren	Moteur de grains	25%
Entrées/Sorties Audio & Pipeline	Callback audio & Buffers temps réel	40%
Tao	Archi multi-thread & Flux audio	50%
Système & Rendu	Pipeline graphique (wgpu)	60%
Julien	Architecture & Normes	100%

Florent

En tant que chef de groupe, mon objectif premier pour cette soutenance était de garantir la mise en place d'une architecture de base inter-modulaire fonctionnelle. Concernant ma partie technique (Analyse Spectrale), l'avancement est conforme au cahier des charges :

- **Algorithme FFT (70 % - Attendu : 70 %) :** L'algorithme de Cooley-Tukey est implémenté en Rust. Il traite correctement les buffers audio et effectue la conversion vers le domaine fréquentiel.
- **Gestion des fenêtres et magnitudes (40 % - Attendu : 40 %) :** Le calcul des magnitudes absolues à partir des nombres complexes est opérationnel. L'implémentation des fonctions de fenêtrage (Hann/Hamming) pour réduire la fuite spectrale (*spectral leakage*) est l'étape actuellement en cours de développement.

Oren

Pour cette première soutenance, les objectifs fixés par le cahier des charges étaient d'atteindre 50 % sur l'architecture du buffer circulaire et 25 % sur le moteur de grains.

L'état d'avancement réel est très positif, puisque je suis en avance sur la planification initiale :

- **Buffer Circulaire (100 % - Attendu : 50 %) :** L'architecture du réservoir de son est entièrement fonctionnelle.
- **Moteur de grains (35 % - Attendu : 25 %) :** Les structures de base (`Grain` et `GrainCloud`) sont opérationnelles.
- **Intégration :** Le lien avec le module d'Entrées/Sorties développé par Tao est établi. Nous parvenons concrètement à récupérer un flux sonore, à le faire transiter par mon moteur granulaire pour le modifier, et à ressortir un son différent.

Grâce à cette avance, nous envisageons d'étendre le périmètre du projet et d'ajouter de nouvelles fonctionnalités (effets supplémentaires ou paramètres de modulation) pour la soutenance finale.

Tao

L'état d'avancement est supérieur aux attentes, avec les résultats suivants :

— **Entrées/Sorties Audio Pipeline (100% - Attendu : 40%) :** Le flux audio de sortie est pleinement fonctionnel. Le périphérique de sortie est détecté et configuré, et le callback audio fournit en continu les échantillons requis par la carte son dans les contraintes du temps réel.

— **Archi multi-thread Flux audio (100% - Attendu : 50%) :** La fréquence de l'oscillateur est pilotable en temps réel via une variable atomique partagée entre le thread principal et le callback, ce qui permet d'ajuster les paramètres sans perturber la continuité du signal audio.

— **Intégration :** Le pipeline mis en place constitue désormais la base commune pour les autres modules : moteurs de synthèse et moteur granulaire peuvent s'y raccorder pour traiter le signal dans un environnement temps réel stable.

Julien

Pour cette partie, les objectifs fixés au début du projet étaient les suivants : concevoir une interface graphique modulaire capable d'afficher un spectre audio en temps réel, proposer un ensemble de contrôles pour la synthèse granulaire (taille de grain, densité, hauteur, etc.) et développer un site web multi-page permettant de présenter le projet et de distribuer les livrables. La planification a été découpée en trois étapes principales : d'abord la mise en place d'une maquette fonctionnelle de la GUI avec des données de test, puis le câblage progressif avec les modules FFT et synthèse, et enfin la réalisation du site web et l'intégration des contenus.

Au moment de la première soutenance, la maquette de l'interface est opérationnelle avec un affichage de spectre simulé et des contrôles interactifs déjà reliés à une structure de paramètres commune. Le site web est également en place, avec la navigation, la charte graphique et la majorité des pages remplies. Les prochaines étapes pour ma partie concernent principalement le branchement définitif de la GUI sur les données FFT en temps réel, les derniers ajustements d'ergonomie, ainsi que la mise à jour du site avec les versions finales de l'application et du rapport.

3 Difficultés & Choix technologiques et Algorithmiques

3.1 Analyse Spectrale - Florent

Le défi principal de mon module a consisté à traduire des concepts mathématiques de traitement du signal en code Rust performant et sécurisé. Afin de monter en compétence, j'ai pris la décision d'implémenter l'algorithme "from scratch" plutôt que de m'appuyer sur des bibliothèques externes existantes (comme `rustfft`).

- **Choix Algorithmique (Cooley-Tukey) :** J'ai opté pour l'algorithme *Radix-2* de Cooley-Tukey. Ce choix permet de réduire drastiquement la complexité temporelle de l'analyse, passant de $\mathcal{O}(N^2)$ pour une transformée classique à $\mathcal{O}(N \log N)$, ce qui est indispensable pour un traitement audio en temps réel à faible latence.
- **Gestion des Nombres Complexes :** Le langage Rust n'intégrant pas nativement les nombres complexes dans sa bibliothèque standard, j'ai dû structurer soigneusement la gestion des types de données pour les calculs en "papillon" de la FFT, en m'assurant d'optimiser les allocations mémoire.
- **Coordination et Workflow :** En tant que chef de groupe, la difficulté a aussi été de synchroniser le développement des quatre membres sur un langage strict comme Rust. La mise en place d'un workflow Git rigoureux nous a permis de lier mon module d'analyse à l'interface de Julien et au pipeline de Tao sans conflit majeur d'intégration.

3.2 Synthèse Granulaire - Oren

Je n'ai pas rencontré de blocages techniques majeurs ou de bugs critiques sur ma partie. La difficulté principale a résidé dans l'apprentissage anticipé du langage Rust et de ses paradigmes. Afin d'avancer sur le projet, j'ai dû me former en autonomie sur des concepts orientés objet appliqués à Rust (l'utilisation des mots-clés `impl`, `Self`, la création de méthodes associées aux structures) avant même qu'ils ne soient abordés en cours. L'enjeu était de comprendre comment "Ruster" mon code pour simplifier et sécuriser la logique algorithmique.

Cette démarche d'apprentissage autonome a directement dicté mes choix technologiques et algorithmiques :

- **Développement "From Scratch" du Buffer Circulaire :** J'ai fait le choix de coder l'intégralité du buffer circulaire à la main plutôt que

d'utiliser une bibliothèque externe (*crate*). Ce choix se justifie par une volonté de maîtrise totale de l'algorithme. C'était le moyen le plus direct et le plus formateur pour comprendre précisément le comportement de la mémoire partagée et de la lecture/écriture par fenêtres glissantes (gestion des indices sécurisée par modulo).

— **Architecture de l'Overlap-Add :** En appliquant les concepts de structures et de méthodes fraîchement appris, j'ai mis en place un système `GrainCloud` qui gère l'apparition dynamique des grains (`spawn_interval`) et leur cycle de vie. L'algorithme actuel effectue la sommation des échantillons de plusieurs grains simultanés pour générer la sortie audio en temps réel.

— **Prochaines étapes (Algorithme de fenêtrage) :** Actuellement, le moteur applique une amplitude constante aux grains. La prochaine évolution algorithmique majeure sera l'implémentation du fenêtrage (application d'une courbe de Hanning ou Gaussienne) pour lisser l'attaque et le relâchement de chaque grain, supprimant ainsi les artefacts sonores ("clics") lors de la superposition.

3.3 Entrées/Sorties Audio & Pipeline - Tao

Les principales difficultés n'ont pas été liées à des bugs bloquants, mais plutôt à la mise en place correcte d'un pipeline audio temps réel respectant les contraintes du callback. L'enjeu central a été de comprendre comment structurer le code autour du modèle de flux imposé par la bibliothèque audio, où le système appelle régulièrement une fonction de rappel pour fournir les échantillons à la carte son. Il a également été nécessaire d'anticiper les questions de synchronisation entre le thread principal (qui pilote les paramètres) et le thread gérant le flux audio, afin d'éviter toute perturbation sonore ou coupure dans le signal. Ces contraintes ont guidé plusieurs choix technologiques et algorithmiques :

— **Utilisation de `cpal` en mode callback :** Le choix de `cpal` permet de travailler directement avec le périphérique de sortie par défaut et d'exposer un modèle basé sur un callback audio. Ce fonctionnement s'aligne avec les exigences du temps réel : la fonction de rappel génère et remplit à la volée les buffers d'échantillons demandés par le système, ce qui impose un code léger, déterministe et sans opérations coûteuses dans cette section.

— **Pilotage des paramètres via variables atomiques :** Pour permettre au thread principal de modifier des paramètres (par exemple la

fréquence de l'oscillateur) sans bloquer le flux audio, une variable atomique partagée (`Arc<AtomicU32>`) est utilisée. Ce choix évite l'emploi de verrous explicites dans le callback et garantit que les mises à jour de paramètres restent sûres et non intrusives pour le temps réel.

— **Intégration d'un buffer circulaire dans le pipeline :** Même si le cœur de la synthèse granulaire est développé dans un autre module, un buffer circulaire a été intégré dès cette étape côté pipeline pour stocker l'historique des échantillons. Cela permet déjà de tester des traitements basés sur le passé du signal (lecture décalée, effets de type delay ou pré granulation) tout en validant le bon fonctionnement de la chaîne Entrée → Traitement → Sortie.

3.4 Système & Rendu - Julien

Pour la partie interface graphique de l'application, mon premier choix a été de structurer clairement les données échangées entre la GUI et le coeur audio. J'ai défini des structures dédiées pour représenter d'une part les trames de spectre issues de la FFT, et d'autre part l'ensemble des paramètres de synthèse granulaire. Cette séparation nette entre données d'affichage et paramètres de contrôle permet de limiter les dépendances entre modules et de faciliter l'intégration progressive des différentes briques du projet. L'interface est organisée autour de deux zones principales : une zone centrale de visualisation du spectre et un panneau latéral de contrôle pour la synthèse. Ce découpage vise à offrir une hiérarchie visuelle claire, où l'utilisateur identifie immédiatement l'information principale (le spectre) et les réglages associés.

Sur le plan de la mise en page, j'ai choisi une disposition en grille simple qui reste efficace, quel que soit le contenu exact du spectrogramme. L'espace d'affichage a été dimensionné pour pouvoir accueillir un spectre d'amplitude ou un spectrogramme en couleurs, en laissant suffisamment de marge pour les légendes et les échelles si nous décidons de les ajouter. Le panneau de contrôle regroupe les paramètres par bloc logique (par exemple, paramètres temporels d'un côté, paramètres liés à la hauteur et à la densité de l'autre), afin de limiter la charge cognitive lors de la manipulation de l'outil. Ces choix de structuration répondent à un double objectif : rendre l'application utilisable rapidement par un utilisateur qui découvre le projet pendant la soutenance et garder une base de code prête à évoluer si l'équipe souhaite ajouter de nouveaux contrôles.

Gestion de la communication avec le moteur audio

Une difficulté importante a été d'anticiper la communication avec le moteur audio temps réel, alors que les différents modules (FFT, synthèse granulaire, pipeline I/O) n'étaient pas encore finalisés au moment de la conception de la GUI. Pour ne pas bloquer l'avancement, j'ai d'abord travaillé avec des données de test : des spectres simulés, mis à jour régulièrement, ce qui m'a permis de valider le comportement visuel de l'interface (rafraîchissement de l'affichage, lisibilité des courbes, réaction des contrôles) sans dépendre du reste du code. Parallèlement, j'ai défini avec les autres membres du groupe une structure de trame de spectre commune (par exemple un vecteur d'amplitudes accompagné de la fréquence d'échantillonnage), qui servira de contrat entre le module FFT et la GUI.

Pour les paramètres de synthèse, j'ai choisi d'exposer depuis l'interface un ensemble réduit mais cohérent de contrôles (taille de grain, densité, hauteur/pitch, éventuellement mix ou largeur stéréo) en m'assurant avec le responsable de la synthèse que les unités et les bornes utilisées étaient compatibles avec le moteur. L'idée était d'éviter à la fois les interfaces trop limitées, qui ne permettent pas d'exploiter le potentiel de la synthèse granulaire, et les interfaces trop complexes, qui deviennent difficiles à comprendre pendant une démonstration courte. Pour le partage effectif des paramètres avec le moteur audio, nous avons envisagé des mécanismes adaptés au temps réel (structures partagées protégées ou canaux non bloquants), de manière à ce que le callback audio ne se retrouve jamais bloqué par la GUI. Même si l'implémentation détaillée reste du ressort des membres en charge de l'audio, ces choix ont guidé la façon dont j'ai conçu l'API de l'interface.

Contraintes de temps réel et ergonomie

Le contexte temps réel impose un certain nombre de contraintes qui influencent directement la conception de l'interface. Par exemple, la fréquence de mise à jour du spectre et la réactivité des contrôles doivent être suffisamment élevées pour que l'utilisateur perçoive l'application comme fluide, sans pour autant surcharger inutilement le processeur ou provoquer des saccades. Pour répondre à cela, j'ai conçu l'affichage de manière à accepter des mises à jour périodiques (par trames successives), plutôt qu'une redessine complète continue non contrôlée. Visuellement, cela se traduit par un spectre qui évolue de façon régulière et lisible, sans scintillement excessif.

Sur le plan ergonomique, j'ai accordé une attention particulière au choix des gammes de valeurs et à la présentation des contrôles. Les curseurs de

paramètres sont accompagnés de labels explicites et, lorsque c'est possible, de valeurs numériques compréhensibles (millisecondes pour la taille de grain, facteur ou demi-tons pour la hauteur, etc.). L'objectif est que, même sans connaître en détail la synthèse granulaire, un utilisateur puisse comprendre intuitivement l'effet de chaque réglage pendant la démonstration. Cette réflexion ergonomique a parfois nécessité d'ajuster les plages de valeurs proposées par défaut, afin de garder l'interface dans une zone d'utilisation "intéressante" où les changements sont audibles mais restent maîtrisés.

Choix techniques pour le site web du projet

En parallèle de l'interface de l'application, j'ai pris en charge la réalisation du site web de présentation du projet. Nous avons fait le choix d'un site statique, réalisé uniquement en HTML et CSS, sans framework ni back-end, pour plusieurs raisons : simplicité de mise en place, robustesse, et absence de dépendances lourdes le jour de la soutenance. La structure du site suit une organisation multi-page classique mais efficace : une page d'accueil présentant le projet de manière synthétique et visuelle, une page détaillant les objectifs et l'architecture, une page dédiée à l'équipe, et une page "Ressources" regroupant les liens de téléchargement (rapport, code source, version lite). Cette organisation répond aux attentes du sujet tout en facilitant la navigation pour le jury.

Pour renforcer la cohérence avec l'interface audio, j'ai opté pour une charte graphique sombre avec des touches de couleur néon rappelant le monde de l'audio et des visualisations de spectre. Le design repose sur des techniques modernes de mise en page en CSS (grilles et flexbox) qui permettent d'obtenir un rendu responsive sans complexité excessive. Les éléments interactifs importants (boutons de téléchargement, liens de navigation) sont mis en avant par des micro-effets visuels (changements de couleur, ombres douces) qui restent légers et compatibles avec des navigateurs variés. Ce choix d'un site statique mais soigné s'inscrit dans la volonté de proposer une vitrine claire et professionnelle du projet, tout en restant en accord avec le temps disponible et le niveau de complexité raisonnable pour un projet de semestre.

Enfin, la mise en place de ce site m'a permis de centraliser les informations produites par l'équipe : description du contexte, rôles de chacun, architecture, mais aussi liens vers les livrables. D'un point de vue pédagogique, ce travail sur le site web complète bien la partie interface graphique de l'application : dans les deux cas, il s'agit de rendre accessible et compréhensible un ensemble de choix techniques, en les présentant dans une forme agréable et structurée. Cette cohérence entre l'outil lui-même (l'application audio) et sa présentation

(le site) participe à la valorisation du projet lors de la soutenance.

4 Bibliographie Détaillée du Projet

La réalisation de ce projet s'appuie sur un corpus documentaire structuré, allant des fondements mathématiques de l'analyse spectrale aux paradigmes modernes de la programmation système.

1. Théorie du Traitement du Signal (Ouvrages Francophones)

- KUNT, Murat. *Traitement numérique des signaux*. Presses polytechniques et universitaires romandes. (L'ouvrage de référence pour comprendre la décomposition de Fourier et l'échantillonnage en langue française).
- COTTET, Francis. *Traitement des signaux et acquisition de données*. Dunod. (Un guide pragmatique sur l'implémentation des algorithmes de filtrage et la transformation de Fourier rapide).
- OKSMAN, Jacques. *Théorie et traitement du signal*. Dunod. (Excellent pour l'étude des fonctions de fenêtrage et la gestion des densités spectrales de puissance).

2. Informatique Musicale et Synthèse

- ROADS, Curtis. *L'audionumérique*. Dunod. (Traduction française de *The Computer Music Tutorial*, considéré comme la "bible" de la synthèse granulaire et du traitement sonore).
- VINET, Hugues. *Le traitement du signal sonore*. Hermès Science. (Analyse approfondie des techniques de transformation spectrale appliquées à la création musicale).

3. Programmation Système et Algorithmique

- BLANDY, Jim et ORENDORFF, Jason. *Programmation en Rust*. O'Reilly (Édition française). (Livre fondamental pour maîtriser la gestion de la mémoire sans ramasse-miettes et le multithreading sécurisé).
- SEDGEWICK, Robert. *Algorithmes en C / C++ / Rust*. Pearson. (Pour l'optimisation des structures de données et l'implémentation efficace de la récursivité dans l'algorithme de Cooley-Tukey).

4. Ressources Numériques et Standards

- Documentation Officielle Rust (*The Book*). `https://doc.rust-lang.org/book/`. (Référence incontournable pour les concepts de *Ownership* et de *Lifetimes* appliqués aux buffers audio).
- CPAL (Cross-Platform Audio Library). `https://docs.rs/cpal/`. (Documentation technique pour la gestion des flux d'entrée/sortie bas niveau sur Windows, macOS et Linux).
- FFTPACK (Documentation historique). *Netlib*. (Étude des implémentations de référence de la FFT pour la validation de nos propres résultats).

Cette bibliographie constitue le socle de nos recherches. Elle garantit que le moteur **SpectraCore** repose sur des bases mathématiques validées par la littérature scientifique tout en exploitant les outils les plus modernes du développement logiciel.